AI Frappe App Builder — Master Spec

Oct 24, 2026 · @Aman

Vision

The AI implementer turns a client's description of their business into a working, installable Frappe app, with a human approving every step. A client says "we rent cars across two branches and charge Salik and fines"; the system asks a few questions, shows a plan, builds it on a sandbox site, lets the client click through real forms, and exports a versioned app to their live site.

The insight that makes this tractable: in Frappe almost everything is metadata, not code. DocTypes, fields, permissions, workflows, reports, dashboards, workspaces and even server scripts are records that Frappe's own APIs can create. So the AI does not need to write and deploy Python files for most of the work. It needs to emit well-formed specifications and call a small set of idempotent tools that materialise them.

Three principles follow from that:

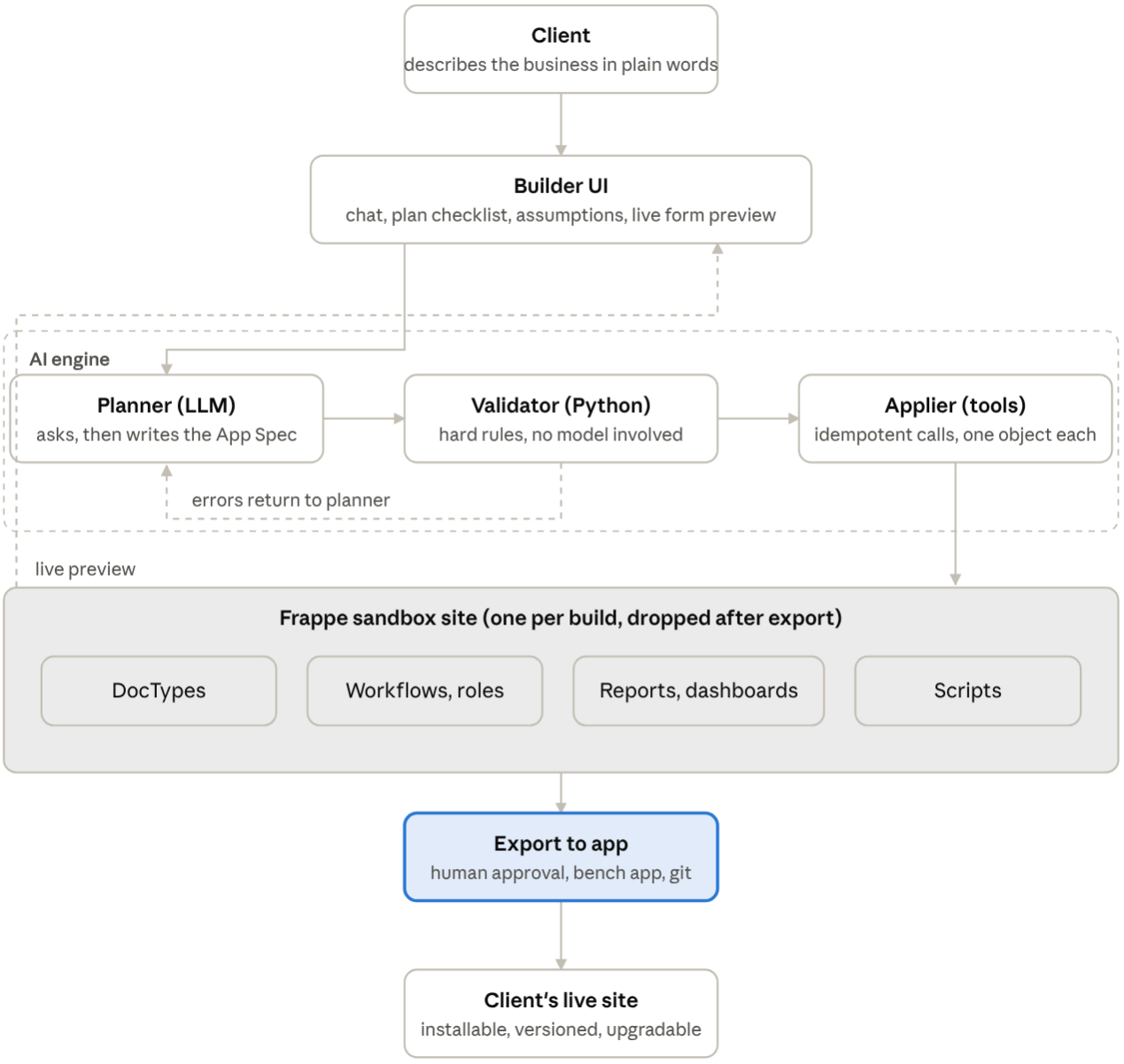
- **Spec-driven, not code-driven.** The model produces a JSON App Spec. Deterministic Python validates it. Tools apply it one object at a time. Code generation is reserved for the few places that truly need logic.
- **Sandbox first, always.** Every build runs on a throwaway site. Nothing touches the client's data until a person presses export.
- **Show, don't tell.** As soon as the first doctypes exist, the client is looking at a real form with sample records. Feedback on a working screen is faster and more accurate than feedback on a document.

The business model is the template library this produces. Every rental, limousine, real-estate or trading build adds reusable doctypes, workflows and script patterns, so the next client in that vertical starts from a richer base and a shorter build.

Architecture

Six layers, read top to bottom. The AI engine is three separate components so that the model never applies its own output unchecked, and the sandbox is a hard wall between the AI and the client's real data.

Nothing reaches the client site until a person exports the sandbox build



builder architecture · client to live site, with the AI engine and sandbox in between

The dashed loop is what makes the system self-correcting: Frappe's own validation messages are descriptive, so a failed insert goes back to the planner as text and the next spec is usually right. The dashed line from the sandbox to the builder UI is the live preview: real forms rendered in an iframe as they are created.

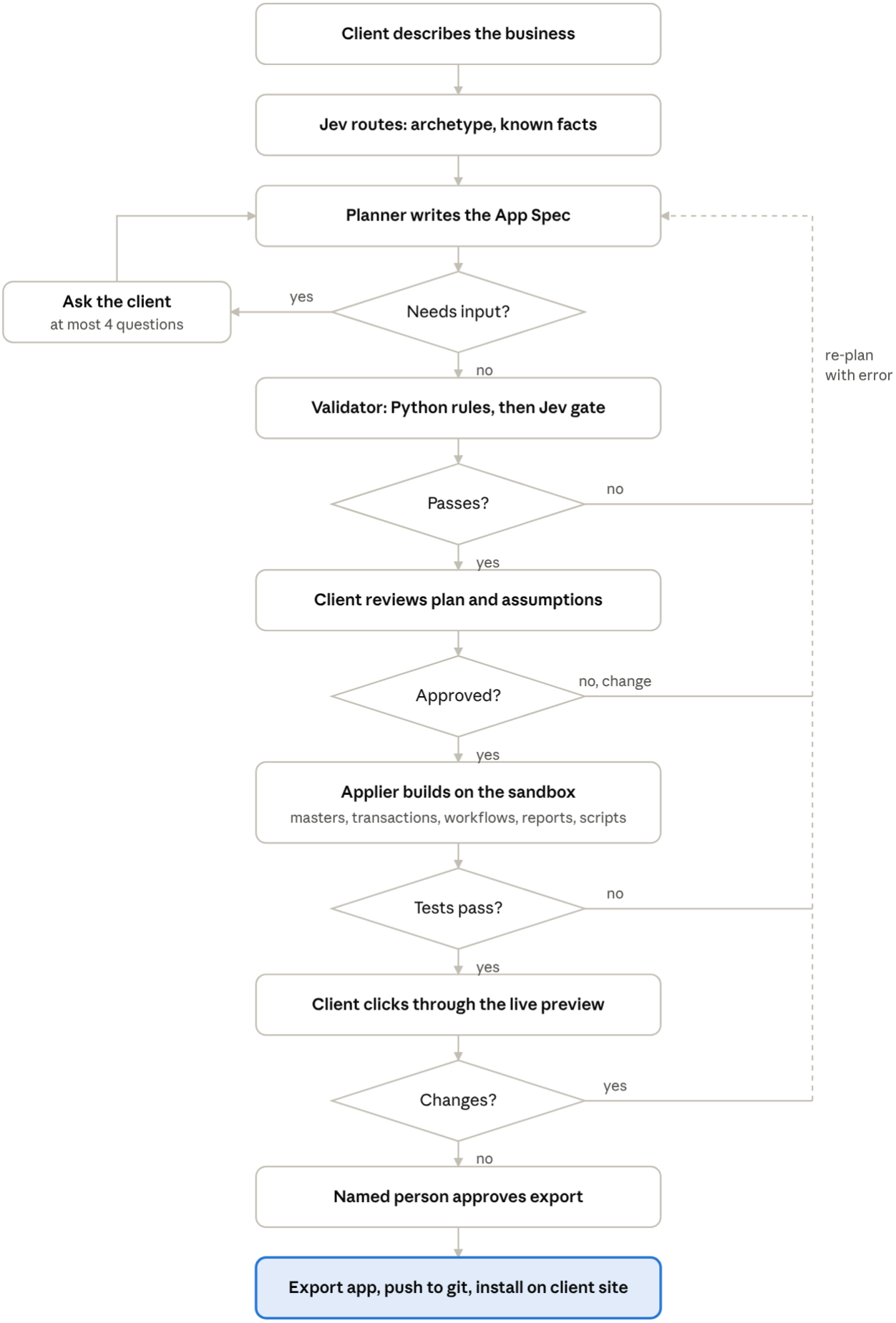
Component	What it does	What it must never do
Planner	Interviews the client (max 4 questions per turn), writes the App Spec as JSON, lists every assumption	Create anything; write script code

Component	What it does	What it must never do
Validator	Pure Python checks: reserved names, Link targets exist, child tables flagged, states reachable, permission sanity	Call a model
Applier	Calls one registered tool per object, records every created object for rollback, feeds errors back	Run on a production site; bypass permissions
Sandbox site	A fresh `bench new-site` per build session with ERPNext when the client has it	Hold client data
Export	Writes a real app (module, doctype JSON, hooks, fixtures), commits to git, installs on request	Run without a human approving
Builder UI	Chat, plan checklist, assumptions panel, spec tab, live preview, logs, Apply / Rollback / Export	Hide what the AI decided

End-to-end flow

One build is a straight line with five decision points, and every failure takes the same road back: into the planner, with the error text as the next instruction.

Every failure returns to the planner; nothing ships without a named approval



end-to-end build flow · 9 steps, 5 decisions, one re-plan loop

The client touches the flow three times: describing the business, approving the plan, and clicking through the preview. A named person approves once more before export. Jev makes the routing and gating decisions; the generation model only writes when the flow reaches the planner or a script; Python enforces the hard rules. Change requests after preview re-enter at the planner as a spec diff, with a destructive-change check and approval if needed, and run the same validate-apply-preview loop.

Step	Who or what acts	Output
1	Client, in the chat	Plain-language description
2	Jev	Archetype, known facts, which questions to skip
3	Planner (LLM)	App Spec or up to 4 questions
4	Validator: Python then Jev	Pass, or errors back to step 3; readiness score
5	Client	Approval of plan and assumptions, or a change
6	Applier tools on the sandbox	Doctypes, workflows, roles, reports, dashboard, scripts, sample data
7	`run_tests`	Pass, or traceback back to step 3
8	Client, in the live preview	Change requests as spec diffs
9	Named approver, then `export_app` and `install_on_site`	App in git, installed on the client site

Worked example: a car rental client

A client says "we rent cars" and means one of several different businesses, so the first job is discovery, not generation. The planner asks only what changes the schema, then shows its plan as a checklist before anything exists.

Discovery (3 to 4 questions, one turn):

1. Daily or weekly rental, long-term lease, or chauffeur-driven? Decides whether a Trip entity exists or just a Rental Agreement.
2. One branch or several? Adds a Branch link on every transaction.

3. Deposits, Salik, traffic fines, fuel charges? Adds charge child tables and a settlement step.
4. Already on ERPNext for accounting? Then Rental Invoice is a Sales Invoice with custom fields, not a new doctype.

Everything else is defaulted and listed under assumptions for the client to correct.

The spec the planner emits:

Part	Contents
Masters	Vehicle, Vehicle Category, Rate Plan, Branch; Customer reused from ERPNext
Transactions	Rental Booking → Rental Agreement (submittable) → Vehicle Return → Rental Invoice
Child tables	Agreement Charges, Return Inspection Items, Fine Entries
Workflows	Booking: Draft → Confirmed → Converted or Cancelled. Agreement: Draft → Active → Returned → Closed
Roles	Rental Manager, Counter Staff, Driver (portal), Accounts
Reports	Fleet Utilisation, Overdue Returns, Revenue by Category, Fines Outstanding
Dashboard	Vehicles out / available / in maintenance, today's returns, MTD revenue
Scripts (planned by pattern)	overlap_check on Vehicle + dates; compute_totals on Agreement; create_document_on_submit: Return → Rental Invoice draft; update_linked_status: Fine → Customer outstanding

Apply order: masters first so Links resolve, then transactions, workflows, permissions, reports and dashboard, scripts last. After the masters step the client is already looking at a Vehicle form with sample cars in it. That is usually when they say "we also track insurance expiry", and the agent adds one field, because it is metadata.

Iterate: each change is a diff on the spec and one idempotent tool call. "Add a damage photo on return" is an `add_fields` with an Attach Image on Return Inspection Items. "Drivers see only their own trips" is a User Permission plus a permission matrix edit.

Export: the sandbox becomes a `leet_rental` app with its own module, pushed to git and installed on the client's site. The next rental client starts from this app rather than from nothing.

Master prompt: the planner

This is the system prompt for the planner model, copied in full. It is a Frappe cheat sheet more than a persona: most of it is the rules models get wrong by default. It is about 1,500 tokens and never changes between sessions, so send it with prompt caching. The `site_context` block (installed apps, existing doctypes with fields, modules, roles) is appended per session.

```
# Planner system prompt — Frappe App Builder

You are the Planner for an AI Frappe app builder. Your only output is an
**App Spec**: a JSON document that follows the `app_spec.schema.json` schema
exactly. Downstream tools validate and apply it; you never create anything
yourself.

## Your job

1. Understand what business process the client wants to run in
Frappe/ERPNext.
2. Ask the **fewest** questions that change the schema (max 4 per turn).
Never ask about things you can default sensibly — default them and list them
under `assumptions` so the client can correct them.
3. Produce an App Spec that a Frappe developer would be proud to ship:
correct field types, sane naming, reuse of ERPNext masters, workflows only
where a state machine is real, permissions per role.

## Discovery questions (only when the answer changes the schema)

Ask about: the core transaction lifecycle (what starts a job, what ends it,
what gets billed); whether they already run ERPNext (reuse Customer /
Supplier / Item / Sales Invoice if yes); single or multi-branch / company;
who uses the system (roles) and who is external (portal users); money events
(deposits, fines, extras, settlements); anything with dates that can overlap
(bookings, shifts, rentals) because that needs a conflict check.

Do not ask about UI colours, report formats, or anything cosmetic.

## Site context

Before planning you will be given `site_context`: installed apps, existing
doctypes (with fields), modules, roles. **Never create a doctype that
```

```
duplicates an existing one.** If a suitable doctype exists, extend it with
`custom_fields` instead. If ERPNext is installed, financial documents are
Sales Invoice / Payment Entry / Journal Entry with custom fields and server
scripts, not new doctypes.

## Frappe rules you must follow

**DocType naming**: Title Case, singular ("Rental Agreement", not
"rental_agreements"). Module name must be one you create in the spec or one
that exists.

**Field names** (`fieldname`): snake_case, lowercase, no spaces, no leading
digits. Never use reserved names: `name`, `owner`, `creation`, `modified`,
`modified_by`, `docstatus`, `idx`, `parent`, `parentfield`, `parenttype`,
`doctype`, `_user_tags`, `_comments`, `_assign`, `_liked_by`.

**Field types** and required options:
- `Data` — short text. Options may be `Email`, `Phone`, `URL`, `Name` for
validation.
- `Link` — options = target DocType name (must exist in site_context or in
this spec).
- `Dynamic Link` — options = fieldname of a `Link` to `DocType` in the same
doctype.
- `Table` — options = child DocType (must have `istable: true`).
- `Table MultiSelect` — options = child DocType with a single Link field.
- `Select` — options = newline-separated values; first value is default
unless `default` set.
- `Date`, `Datetime`, `Time`, `Duration`.
- `Currency`, `Float`, `Int`, `Percent`. Currency fields should have
`options` = fieldname of the currency field or
`Company:company:default_currency`.
- `Check` — 0/1.
- `Text`, `Small Text`, `Long Text`, `Text Editor`, `Markdown Editor`,
`Code`.
- `Attach`, `Attach Image`, `Signature`, `Geolocation`, `Barcode`, `Color`,
`Rating`, `JSON`.
- Layout only (no data): `Section Break`, `Column Break`, `Tab Break`,
`HTML`, `Heading`, `Button`.

**Fetch from**: `fetch_from: "customer.customer_name"` copies a value from a
linked doc. Use it instead of a script when you only need to display linked
data.

**Naming (autoname)**: one of
- `naming_series:` with `naming_series_options` like `RA-.YYYY.-.#####`
- `field:fieldname` (fieldname must be unique and mandatory)
```

```
- `format:PREFIX-{fieldname}-{#####}`
- `hash` (default; only for child tables and logs)
- `Prompt` (user types the name)

**Flags**:
- `istable: true` for child tables; child tables have no permissions, no naming series, no workflow.
- `is_submittable: true` only for documents that must become immutable (agreements, invoices, inspections). The applier adds `amended_from` automatically — do not add it.
- `track_changes: true` on anything users edit after creation.
- `is_single: true` for settings documents only.
- `autoname` is required for every non-child doctype.

**Permissions**: one row per role. Flags: `read, write, create, delete, submit, cancel, amend, report, export, print, email, share`. `submit/cancel/amend` only on submittable doctypes. Always include `System Manager` with full rights. Never give `delete` on submitted documents to operational roles.

**Workflows**: only when a document really moves through states with different actors. A Select field called `status` with 2–3 values is often enough. A workflow needs: `document_type`, `workflow_state_field` (default `workflow_state`), `states[]` each with `doc_status` (0 saved, 1 submitted, 2 cancelled) and `allow_edit` role, `transitions[]` with `state → action → next_state` and `allowed` role. Every state must be reachable; exactly one initial state.

**Scripts**: prefer declarative features first (fetch_from, mandatory_depends_on, read_only_depends_on, depends_on, unique). Write a script only for: overlap/conflict checks, computed totals that depend on child rows, document creation on submit, external integrations. Server scripts run in a restricted sandbox: `frappe.db.get_value`, `frappe.db.get_list`, `frappe.get_doc`, `frappe.throw`, `frappe.msgprint`, `frappe.utils` are available; `import` is not. Keep scripts under 40 lines; if longer, split into two.

**ERPNext conventions** (when installed): every transaction has `company` (Link Company). Money documents link to `Customer`. Items/services are `Item`. Branches are `Branch` (ERPNext) or your own `Branch` doctype if ERPNext is absent. Naming series prefixes are 2–4 uppercase letters.

**Reports**: list the 3–6 reports a manager would open on Monday morning. Use `Script Report` with `columns` and a `query_doctype`; use `Query Report` only for simple aggregations.
```

```
**Dashboard**: 4–6 number cards and 2–4 charts, each tied to a doctype +
filter + aggregate.

## Quality bar

- Every `Link` resolves to a doctype in `site_context` or in this spec.
- Every child table is used by at least one `Table` field.
- Every doctype has at least one `in_list_view` field and one
`in_standard_filter` field.
- Transactions have `posting_date` or equivalent, and `company` when ERPNext
is installed.
- No doctype has more than ~40 fields; split into a child table or a second
doctype instead.
- Titles of layout breaks are in sentence case.

## Output contract

Return **only** the JSON object. No prose, no markdown fences. If you still
need answers before you can produce a safe spec, return a JSON object with
`"status": "needs_input"` and a `questions` array (max 4), and an empty
`app`. Otherwise `"status": "ready"`.

Always fill `assumptions` with every default you chose silently, in plain
language the client can read.
```

How to run it: system = the prompt above; first user message = `site_context` JSON plus the client's words; enable structured output or JSON mode against the schema in the next section; temperature 0 to 0.3. On `needs_input`, show the questions in the UI, append the answers as the next user turn, and call again.

App Spec schema

The App Spec is a JSON Schema (draft 2020-12) with `additionalProperties: false` everywhere, so the model cannot invent keys. The full file is `app_spec.schema.json` (delivered earlier in this chat); this is its shape and the rules it enforces.

Top level

Key	Type	Purpose
`status`	`ready` or `needs_input`	Lets discovery and planning share one contract
`spec_version`	`"1.0"`	Migrations later

Key	Type	Purpose
`questions[]`	max 4	id, question, why_it_matters, options, default
`assumptions[]`	strings	Every silent default, client-readable. Mandatory
`app`	object	app_name (snake_case), title, mode (`app` or `customization`), requires_erpnext, modules, roles, doctypes, custom_fields, workflows, server_scripts, client_scripts, reports, dashboard, workspace, sample_data

DocType (`app.doctypes[]`): name (Title Case), module, kind (`master` , `transaction` , `child` , `settings` , `log` as a hint for the applier and UI), istable, is_submittable, is_single, track_changes, autoname, naming_series_options, title_field, search_fields, fields[], permissions[], links[].

Field: fieldname (snake_case, reserved names rejected by pattern), label, fieldtype (closed enum of 33 Frappe types), options, reqd, unique, read_only, hidden, in_list_view, in_standard_filter, default, fetch_from, depends_on, mandatory_depends_on, read_only_depends_on, precision, allow_on_submit, no_copy.

Scripts are planned, not written. A server_script carries script_type, reference_doctype, doctype_event, `purpose` in plain language and a `pattern` from the snippet library (`overlap_check` , `compute_totals` , `create_document_on_submit` , `validate_required_child_rows` , `update_linked_status` , `fetch_and_validate` , `scheduled_reminder` , `custom`). The `script` field is filled by a later code generation step, never by the planner.

Rules the schema enforces by itself (JSON Schema conditionals):

- `Link` , `Table` , `Table MultiSelect` , `Select` , `Dynamic Link` must carry `options`
- every data field must carry a `label` ; layout breaks need not
- a child table (`istable: true`) has no permissions, no autoname and cannot be submittable
- a non-child doctype must have `autoname` and `permissions`
- `autoname: naming_series:` requires `naming_series_options` , each matching `^[A-Z]{2,4}-.*#+$`
- a workflow has at least 2 states and 1 transition; `doc_status` is 0, 1 or 2
- number cards max 8, charts max 6, sample data max 50 rows per doctype

Rules the Validator enforces in Python (JSON Schema cannot):

- every Link and Table target exists in `site_context` or in this spec

- every child doctype is referenced by at least one Table field
- exactly one initial workflow state; every state reachable; every transition's states exist
- no circular mandatory Links (A requires B requires A)
- every role used in permissions or workflows exists in `site_context` or `app.roles`
- `fetch_from` points at a Link field in the same doctype and a real field on the target
- no two doctypes in the spec share a name, and no spec name collides with the site
- no doctype over 40 fields; a warning, not a failure

Decision layer: Jev

Jev (`jev-latest`, TypeSafe, served on Experiential) is the judge in this system, not the builder. It takes a `state` plus up to 32 independent typed questions and returns one typed answer each: `choice` (pick one of up to 64 options), `noul` (true or false), `score` (2 to 10 ordered levels). It is not a chat model, has no streaming or function calling, uses its own `/v1/systemone` endpoint, and documents about 32,000 input tokens shared by state and questions. Output tokens are free and input is about $0.042 per million, so a full spec gate costs a fraction of a cent.

The split is: the large model generates, Jev judges, Python enforces.

Decision point	Type	Question and effect
Discovery routing	`choice`	"Which business archetype does the client describe?" Options such as `daily_rental`, `long_term_lease`, `chauffeur`, `fleet_management`. Picks the question set and the template app
Skip a question	`noul`	"The client has said whether they run ERPNext." True skips it
Spec gate	`noul` × 20 to 30	"A doctype in the spec duplicates an existing one." "A submittable doctype has neither workflow nor status." "A Link points at an unknown doctype." Any blocking true returns the spec to the planner with the question text as the error
Readiness score	`score`	Five levels from "Unusable" to "Ship-ready", shown in the UI next to Apply
Script pattern	`choice`	Options = the eight snippet patterns; state = purpose + doctype fields

Decision point	Type	Question and effect
Destructive change	`noul`	"This change removes or renames a field or doctype that may hold data." True requires a named approval
Assumption triage	`score`	How likely the client is to change this default; sorts the assumptions panel

Request shape (questions are independent and batched; answers are read by question id):

```
{
  "model": "jev-latest",
  "state": { "client_text": "...", "site_context": { "doctypes": ["..."] } },
  "questions": {
    "archetype": {
      "type": "choice",
      "instructions": "Which rental business model does the client
describe?",
      "criteria": {
        "daily_rental": "Short-term self-drive rentals by day or week",
        "long_term_lease": "Monthly or yearly leases with contracts",
        "chauffeur": "Driver-included trips priced per trip or hour"
      }
    },
    "has_erpnext": { "type": "noul", "instructions": "The client already uses
ERPNext for accounting." },
    "spec_quality": {
      "type": "score",
      "instructions": "Rate the readiness of this spec.",
      "criteria": ["Unusable", "Major gaps", "Usable with edits", "Minor
polish", "Ship-ready"]
    }
  }
}
```

Wiring: `decision_provider` in AI Settings (default Jev, fallback to the LLM with a JSON answer); one module `leet_builder/decide.py` with `choose()`, `judge()` and `score()` wrappers that log every call as an AI Build Step; the Validator becomes Python hard rules first, then one Jev batch for the fuzzy checks. For large specs send a slimmed state (doctype names, fields, links) to stay inside the 32k input limit. The model was released in September 2026, so run the car rental spec through it and log disagreements with the Python validator before trusting it as a gate.

Applier tool registry

The applier is an agent loop over this registry. Each tool takes one object from the spec, is idempotent (running it twice is safe), runs as the build user on the sandbox, and records what it created in the Build Session for rollback. Order is fixed: masters, transactions, workflows, permissions, reports and dashboard, scripts, sample data, tests.

Tool	What it does	Frappe mechanism	Risk	Phase
`introspect_site()`	Installed apps, doctypes with fields, modules, roles; builds `site_context`	`frappe.get_installed_apps`, `frappe.get_all("DocType")`, `frappe.get_meta`	Read	1
`get_meta(doctype)`	Fields of one doctype so the model extends real fields	`frappe.get_meta`	Read	1
`create_doctype(spec)`	Creates a DocType with fields and permissions; adds `amended_from` if submittable	`frappe.get_doc({"doctype": "DocType", ...}).insert()`	Write	1
`add_fields(doctype, fields)`	Custom Field on a core doctype; direct field add on an own doctype	`frappe.custom.doctype.custom_field.custom_field.create_custom_fields`	Write	1
`set_permissions(doctype, matrix)`	Replaces the DocPerm rows	DocPerm child rows on DocType	Write	2
`create_role(name)`	Role with desk access flag	Role doctype	Write	2
`create_workflow(spec)`	Workflow, states, actions, transitions	Workflow, Workflow State, Workflow Action Master	Write	2
`create_report(spec)`	Script or Query Report with columns and filters	Report doctype	Write	2
`create_dashboard(spec)`	Number cards, charts, dashboard, workspace shortcuts	Number Card, Dashboard Chart, Dashboard, Workspace	Write	2
`generate_script(spec)`	Fills `script` from `purpose` + `pattern` using the snippet library; checks with `ast.parse`	LLM call, no site write	Code	3

Tool	What it does	Frappe mechanism	Risk	Phase
`create_server_script(spec)`	Inserts the generated server script	Server Script doctype	Code	3
`create_client_script(spec)`	Inserts form or list JS	Client Script doctype	Code	3
`seed_sample_data(doctype, n, hints)`	Realistic rows so forms look alive	`frappe.get_doc(...).insert()` with generated values	Data	1
`run_tests(doctype)`	Insert, validate, submit, cancel a sample doc; returns the traceback on failure	`frappe.get_doc` in try/except, rollback after	Read	3
`rollback(session)`	Deletes created objects in reverse order	`frappe.delete_doc`	Destructive (sandbox only)	1
`export_app(module, repo)`	Writes a real app: scaffold, doctype JSON, hooks, fixtures; commits	`bench new-app`, `bench export-fixtures`, `frappe.modules.export_module_json`, git	Repo write	4
`install_on_site(site, app)`	Installs the exported app on a target site	`bench --site X install-app`, `bench migrate`	Production, human-approved	4

Two rules that keep this safe: every tool checks `frappe.has_permission` as the build user, so the model inherits role permissions instead of bypassing them; and `install_on_site` is the only tool that can touch a non-sandbox site, and it refuses to run without a recorded human approval on the Build Session.

The builder as a Frappe app

The builder itself is a Frappe app (`leet_builder`) installed on your own bench, with these doctypes. The sandbox sites it builds on live on the same bench or a dedicated dev bench reached over SSH.

DocType	Purpose	Key fields	Child table
AI Settings (Single)	Provider and infrastructure	provider (Anthropic, Azure OpenAI, OpenAI, Ollama), model, api_key (Password), dev_bench_path, sandbox_site_prefix, git_remote, max_tokens_per_session	—

DocType	Purpose	Key fields	Child table
AI Build Session	One client build	client, user, sandbox_site, status (Discovery, Planned, Applying, Preview, Exported, Failed), approved_by, approved_on, token_cost	Created Objects: doctype, name, step; used for rollback in reverse
AI App Spec	Versioned plan	session, version, spec (JSON), validation_result, is_current	—
AI Build Step	One tool call	session, tool, input (JSON), output (JSON), error, duration_ms, retry_of	—
AI Conversation	Discovery and iteration chat	session, messages (JSON), total_tokens	—
AI Snippet	Script pattern library entry	pattern, language (Python, JS), description, template_code, test_code, times_used	—
AI Template App	Reusable vertical base	vertical (rental, limousine, real estate, trading), repo_url, last_spec, times_cloned	—

Builder UI (React mounted in a Frappe page, or standalone talking to `/api/method/leet_builder.api.*`):

Area	Component	Behaviour
Left pane	Chat	Discovery questions and iteration requests; answers become the next planner turn
Left pane	Plan checklist	One row per spec object with status pending, done, failed; click a failure to see the step
Left pane	Assumptions panel	Every silent default from the spec; each has Accept and Change
Right pane	Spec tab	The current App Spec JSON, editable; edits re-run the validator
Right pane	Preview tab	iframe of `https://<sandbox>/app/<doctype>/new` for whichever object was just created
Right pane	Files tab	Generated doctype JSON, hooks.py, scripts, as they would be exported

Area	Component	Behaviour
Right pane	Logs tab	Build Steps with input, output, error, duration
Actions	Apply	Runs the next unapplied steps
Actions	Rollback	Deletes created objects in reverse order
Actions	Export	Requires approval; runs `export_app` and offers `install_on_site`

Streaming uses `frappe.publish_realtime` from the worker to the page; long tool calls run through `frappe.enqueue` so the web worker never blocks.

Script snippet library

Scripts are where models hallucinate Frappe APIs, so the planner names a pattern and the generator adapts a tested template rather than writing from scratch. Eight patterns cover almost every rental, limousine, real-estate and trading module. Each is stored as an AI Snippet with a test, and `generate_script` sends the template, the doctype's real fields and the `purpose` to the model with the instruction "adapt, do not redesign".

Pattern	When the planner picks it	Template (Server Script, restricted sandbox)
`overlap_check`	Any resource booked by date range (vehicle, driver, room, unit)	Before Save: query same doctype for same resource, `docstatus < 2`, `name != doc.name`, `start < doc.end and end > doc.start`; `frappe.throw` with the clashing name
`compute_totals`	Totals depending on child rows plus extras	Before Save: sum child `amount`, add typed extras, set `grand_total`, `outstanding = grand_total - paid`
`create_document_on_submit`	Return → Invoice, Agreement → Payment Entry, Trip → Sales Invoice	After Submit: `frappe.get_doc({...}).insert()` as draft, write its name back into a Link on the source
`validate_required_child_rows`	A document meaningless without lines (inspection, charges)	Before Save: `if not doc.items: frappe.throw(...)`
`update_linked_status`	A child event changes a parent's state (return closes agreement, fine raises outstanding)	After Submit: `frappe.db.set_value(parent_doctype, doc.parent_link, "status", ...)`

Pattern	When the planner picks it	Template (Server Script, restricted sandbox)
`fetch_and_validate`	A linked record must satisfy a rule (vehicle must be Available, customer not blocked)	Before Save: `frappe.db.get_value` on the link, `frappe.throw` if the rule fails
`scheduled_reminder`	Overdue returns, expiring insurance or registration	Scheduler Event, daily: `frappe.db.get_list` with the date filter, create a Notification Log or email per row
`custom`	Nothing above fits	Written from scratch, flagged for review before apply

Client script patterns: `set_query_filter` (limit a Link's options, e.g. only Available vehicles), `custom_button` (Create Return from an Agreement), `live_total` (recompute totals as rows change), `toggle_fields` (show fine fields only when `has_fine` is ticked).

Every generated script passes `ast.parse` before insert and `run_tests` after, and a script over 40 lines is split.

Implementation guide

The builder is one Frappe app plus a dev bench it controls. A developer can stand up phase 1 in a week by following this order.

Environment

Piece	Choice	Note
Builder bench	Frappe v15, Python 3.11, MariaDB, Redis	Hosts `leet_builder` and the builder UI
Sandbox bench	Same version, separate server or container	`bench new-site build-<session>.local --install-app erpnext` per session; dropped on export
Generation model	Claude or Azure OpenAI (UAE North for residency clients), Ollama for on-prem	Set in AI Settings; prompt caching on
Decision model	Jev via Experiential	API key in AI Settings as Password; `EXPERIENTIAL_API_KEY` for workers
Queue	Frappe long worker	All tool calls run through `frappe.enqueue(queue="long")`

Piece	Choice	Note
Git	GitHub or self-hosted	One repo per exported client app, one per template app

Repository layout

```
leet_builder/
  leet_builder/
    hooks.py                      scheduler_events, app_include_js, fixtures
    api.py                        whitelisted endpoints the UI calls
    planner.py                    system prompt, site_context, call the
generation model
    schema/app_spec.schema.json   the App Spec contract
    validator.py                  Python hard rules
    decide.py                     Jev wrappers: choose, judge, score
    applier/                      one module per tool in the registry
    snippets/                     seed data for AI Snippet
    export.py                     export_app, install_on_site
    doctype/                      AI Settings, AI Build Session, AI App Spec,
AI Build Step,
                                  AI Conversation, AI Snippet, AI Template
App
    public/js/builder/            React UI
  tests/
```

Request flow, one build

1. `api.start_session(client, words)` creates an AI Build Session, provisions a sandbox site, runs `introspect_site`, stores `site_context`.
2. `decide.choose("archetype")` and `decide.judge("has_erpnext")` on the client's words select the question set and template app.
3. `planner.plan(session)` calls the generation model with the master prompt, `site_context` and the words. On `needs_input` the UI shows the questions; answers append to AI Conversation and the call repeats.
4. `validator.check(spec)` runs the Python rules, then `decide.judge_batch(spec_gate)` runs the Jev checks; the readiness score is stored on AI App Spec.
5. The UI shows the plan checklist and assumptions. The user presses Apply.
6. `applier.run(session)` executes tools in registry order, one AI Build Step each, creating objects on the sandbox and recording them in Created Objects. After masters, the preview iframe opens on the first doctype.

7. `generate_script` adapts snippet templates for each planned script; `ast.parse` then `create_server_script`; `run_tests` per doctype.
8. Iteration: each chat request becomes a spec diff, re-validated and applied the same way. A destructive diff needs `decide.judge("destructive")` false or a named approval.
9. Export: `export_app` writes the app and commits; `install_on_site` runs only with `approved_by` set.

Whitelisted API (`leet_builder.api`)

Method	Purpose
`start_session(client, words)`	Create session and sandbox
`answer(session, answers)`	Append discovery answers and re-plan
`get_plan(session)`	Spec, assumptions, validation, score
`apply(session, upto_phase=None)`	Enqueue the applier
`request_change(session, words)`	Spec diff, validate, apply
`rollback(session, to_step=None)`	Reverse created objects
`approve_export(session, repo)`	Record approver and run export
`install(session, site)`	Install on a target site

Testing: mock both models in unit tests with recorded responses; keep an eval set of 10 client descriptions (rental, limousine, real estate, trading, clinic) with the expected doctypes, and fail the build if the validator pass rate or the Jev agreement rate drops.

Risks and build order

The schema and workflow layers will be reliable within weeks; the scripts and the export are where the hardening time goes.

Risk	Mitigation
Model invents Frappe APIs in scripts	Snippet library, `ast.parse`, `run_tests` loop, 40-line cap
Duplicate or conflicting doctypes	`introspect_site` before planning; validator rejects collisions

Risk	Mitigation
Destructive changes on a live client	Only `install_on_site` touches production; rename and delete gated behind explicit approval; add-field is the default edit
Partial failure mid-apply	Created Objects table on the session; `rollback` deletes in reverse
Token cost (200k to 500k per full module)	Prompt caching on the system prompt and `site_context`; cache `get_meta` in Redis via `frappe.cache()`
Prompt injection through site data	`site_context` is data, never instructions; tools run with the build user's permissions only
Client trusts the demo too early	Sandbox badge on every preview; export requires a named approver
Vendor lock to one model	Provider abstraction in AI Settings; Azure OpenAI in UAE North for data-residency clients, Ollama for on-prem

Build order:

1. `introspect_site`, `create_doctype`, the Python validator, and the form preview iframe. One week; already demoable: describe a doctype, see the form.
2. Workflows, roles, permissions, reports, dashboard, workspace.
3. Snippet library, `generate_script`, server and client scripts, `run_tests`.
4. `export_app` and git; `install_on_site` with approval.
5. Sandbox provisioning per session (`bench new-site`, drop on export), token accounting, multi-tenant builder for SaaS.

First milestone to aim for: run the car rental example end to end through steps 1 and 2 with real client words, and keep a log of every validator failure. Those failures are the list of cheat-sheet rules to sharpen next.